

1. Environment Overview: AWS Elastic Beanstalk (PaaS)

For the final project, **AWS Elastic Beanstalk** was selected as the core deployment platform. As a Platform as a Service (PaaS) solution, it provides an orchestrated environment that streamlines the deployment process while maintaining high availability and security.

Technical Implementation Details

Elastic Beanstalk was configured to automate and manage the following infrastructure components:

- **Compute:** Provisioning of **Amazon EC2** instances tailored for the application workload.
- **Runtime Environment:** Fully managed execution environment using **Python 3.13**.
- **Database Integration:** Seamless connectivity with **Amazon RDS (PostgreSQL)**, ensuring data persistence and managed backups.
- **Traffic Management:** Deployment of an **Application Load Balancer (ALB)** to efficiently distribute incoming traffic across multiple instances.
- **Elasticity:** Integrated **Auto Scaling** group that adjusts capacity based on real-time **Amazon CloudWatch** metrics.
- **Observability:** Comprehensive logging and health monitoring via **Amazon CloudWatch** logs and dashboards.

2. Technology Rationale & Architectural Decisions

The selection of the AWS stack for this project was driven by the need for a balance between **rapid deployment**, **high availability**, and **robust security**. Below is the justification for the core technologies implemented.

Why AWS Elastic Beanstalk (PaaS)?

While manual EC2 management offers maximum control, Elastic Beanstalk was chosen for its operational efficiency:

- **Deployment Simplicity:** Enables a streamlined "Code-to-Cloud" workflow where a single application archive triggers a full environment orchestration.
- **Deep AWS Ecosystem Integration:** Out-of-the-box compatibility with **Amazon RDS**, **CloudWatch**, and **IAM**, ensuring that all components communicate securely and efficiently.
- **Managed Scalability:** Automates the complexity of scaling and updates, reducing the risk of downtime during traffic spikes or version transitions.
- **Reduced Operational Overhead:** Eliminates the need for manual patching and server provisioning, allowing focus on application-level security and logic.

Security and Scalability Framework

The environment was architected with a "**Security First**" mindset, adhering to the following principles:

- **Network Isolation:** Deployment within a custom **Amazon VPC**, utilizing private subnets for sensitive resources (like RDS) to prevent direct internet exposure.
- **Granular Access Control:** Implementation of **IAM Roles** and **Security Groups** to enforce the *Principle of Least Privilege (PoLP)*, ensuring only necessary traffic and permissions are allowed.
- **High Availability:** The combination of an **Application Load Balancer (ALB)** and **Auto Scaling** ensures the application can adapt to load changes while maintaining a high uptime SLA.

Educational Note: The Advantage of PaaS in Cybersecurity

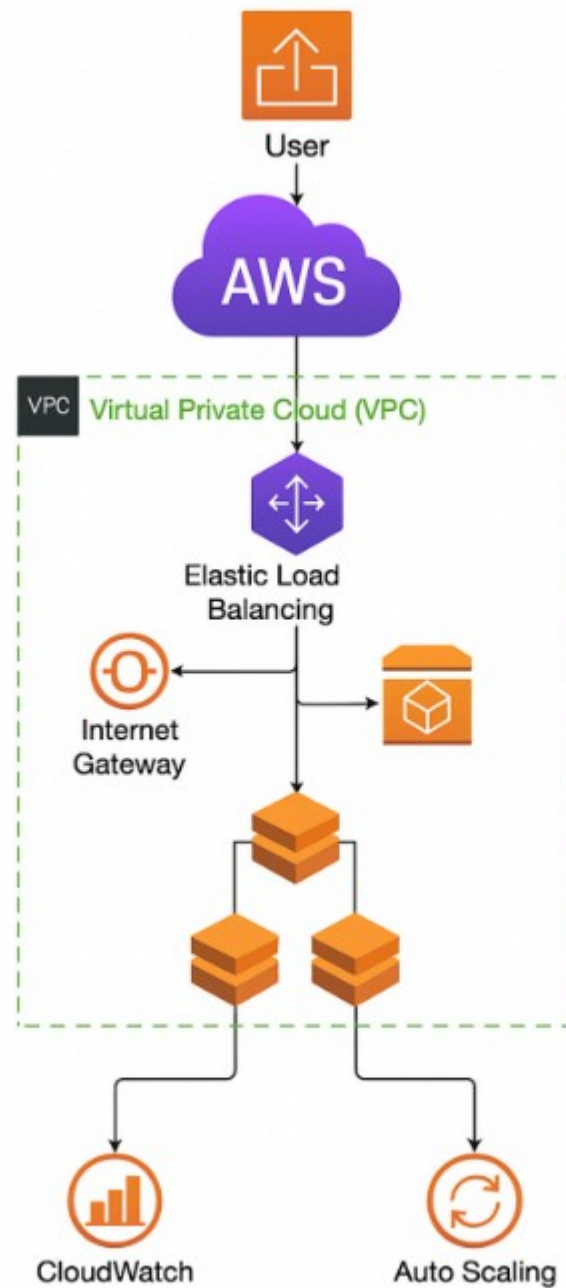
From a cybersecurity perspective, using a managed service like Elastic Beanstalk reduces the **attack surface**. By offloading the management of the underlying OS and runtime patches to AWS, the risk of vulnerabilities arising from unpatched infrastructure is significantly mitigated.

3. Solution Architecture

The following diagram illustrates the cloud infrastructure designed for this application. It represents a highly available, secure, and scalable web environment within the AWS ecosystem.

Architectural Flow & Components

- **Entry Point:** The **User** accesses the application via the public internet, where requests are first handled by the AWS Cloud boundary.
- **Virtual Private Cloud (VPC):** All compute resources are isolated within a dedicated **VPC**, providing a logical network perimeter.
- **Traffic Distribution (ELB):** An **Elastic Load Balancer** acts as the single point of contact. It intelligently routes incoming traffic to healthy EC2 instances across multiple Availability Zones, preventing any single point of failure.
- **Connectivity (Internet Gateway):** The **Internet Gateway (IGW)** allows communication between the VPC and the internet, enabling the Load Balancer to receive external requests.
- **Compute Layer:** The application runs on **EC2 instances** managed by Elastic Beanstalk. These instances are organized in a cluster to ensure workload distribution.
- **Operational Intelligence:**
 - **Amazon CloudWatch:** Continuously monitors system health, performance metrics, and logs.
 - **AWS Auto Scaling:** Dynamically adjusts the number of EC2 instances based on CloudWatch alarms, ensuring the application stays responsive during traffic spikes and cost-effective during low demand.



4. Deployment and Configuration Guide

This section outlines the end-to-end process of deploying the Flask application to **AWS Elastic Beanstalk**, including database integration, security hardening, and monitoring setup.

Phase 1: Application Source Preparation

Before deployment, the application environment was standardized to ensure compatibility with AWS Linux 2023:

- **Core Logic:** Developed a **Flask** application (`application.py`) with **SQLAlchemy** for ORM-based interaction with PostgreSQL.
- **Dependency Management:** Generated a `requirements.txt` file to ensure a consistent runtime environment.
- **Process Management:** Created a `Procfile` to instruct Elastic Beanstalk on how to launch the application using the **Gunicorn** WSGI server:
`Makefile`
`web: gunicorn application:app`
- **Artifact Creation:** All essential files were bundled into a high-integrity **ZIP archive** for deployment.

Phase 2: Orchestrating the Environment

The deployment was executed through the AWS Management Console with the following specifications:

1. **Environment Type:** Web Server Environment.
2. **Platform Stack:** Python 3.13 running on **Amazon Linux 2023**.
3. **Deployment Method:** Manual upload of the prepared ZIP artifact.

Phase 3: Secure Database Integration (RDS)

To ensure data persistence and security, an **Amazon RDS (PostgreSQL)** instance was provisioned:

- **Network Security:** Configured **Security Groups** to allow inbound traffic only from the Elastic Beanstalk instances (limiting the attack surface).
- **Environment Secrets:** Instead of hardcoding credentials, sensitive data (`DB_HOST`, `DB_USER`, `DB_PASSWORD`) was injected via **Elastic Beanstalk Environment Properties**.
- **Code Integration:** The application retrieves these credentials securely using `os.environ` for database URI construction.

Phase 4: Security Architecture & IAM

Adhering to the **Principle of Least Privilege (PoLP)** and network segmentation:

- **VPC Design:** Implemented a multi-tier architecture where the **Elastic Beanstalk** environment resides in public subnets, while the **RDS Database** is isolated in private subnets.
- **IAM Roles:** Created a dedicated **EC2 Instance Profile** (IAM Role) that grants the minimum necessary permissions for instances to communicate with RDS and push logs to CloudWatch.

Phase 5: Monitoring, Scalability, and High Availability

The infrastructure is designed to be self-healing and observable:

- **Observability:** Integrated **Amazon CloudWatch** to track `CPUUtilization` and `EnvironmentHealth`.

- **Alerting:** Configured a **CloudWatch Alarm** integrated with **Amazon SNS** to trigger notifications if CPU load exceeds 80%.
- **Elasticity:** Configured **Auto Scaling** with a dynamic range (Min: 1, Max: 4 instances) based on real-time traffic demands.
- **Load Balancing:** Deployment of an **Application Load Balancer (ALB)** to distribute traffic and perform health checks on the underlying fleet.

5. Implementation Evidence & Monitoring

This section provides visual confirmation of the successfully deployed infrastructure, its operational health, and the security/cost controls implemented.

5.1 Deployment & Connectivity

- **Application Environment:** The Elastic Beanstalk dashboard confirms the environment `Flask-app-env` is in a **Green (Healthy)** state, running on **Python 3.11/Amazon Linux 2023**.
- **Database Connectivity:** A successful browser test confirms that the application is correctly interfaced with the **Amazon RDS (PostgreSQL 17.2)** instance, retrieving version information via SQLAlchemy.
- **Infrastructure:** The **Application Load Balancer (ALB)** is active and correctly mapping traffic across multiple availability zones.

Environment Dashboard confirming health and platform version.

Live application URL showing successful handshake with the RDS instance.

5.2 Scalability & Monitoring

- **Auto Scaling Configuration:** The system is configured for high availability with a dynamic instance range (1 to 3 instances) and an automated scale-up trigger when CPUUtilization reaches **80%**.
- **CloudWatch Alarms:** A custom alarm (`flask-alarm`) is active, monitoring the fleet's CPU usage.
- **Proactive Notifications:** Integrated **Amazon SNS** topic (`Default_CloudWatch_Alarms_Topic`) ensures that any system stress triggers an immediate email alert to the administrator.

Visual monitoring of CPU metrics with an active threshold alarm.

SNS Subscription confirmed for real-time security and performance alerts.

5.3 Cost Optimization (AWS Budgets)

To ensure fiscal responsibility and prevent "cloud sprawl," **AWS Budgets** has been implemented:

- **Monthly Budget:** Set at a strict **\$5.00** threshold.
- **Tiered Alerting:** Three distinct alerts are configured:
 1. **85% of Actual Costs:** Early warning of budget approach.
 2. **100% of Forecasted Costs:** Predictive alert based on usage patterns.

3. **100% of Actual Costs:** Critical notification when the limit is reached.

Budget dashboard showing cost tracking and threshold status.

5.4 Advanced Monitoring & Incident Response

To demonstrate the practical application of AWS monitoring tools, a stress test was simulated, triggering the configured security and performance protocols.

- **Real-time Incident Detection:** The CPUUtilization alarm transitioned to the **ALM (In alarm)** state as traffic exceeded the 80% threshold.
- **Visualizing Performance:** The CloudWatch metrics graph clearly shows the upward trend in resource consumption, confirming that the monitoring system accurately reflects the environment's real-time state.
- **Automated Notification Loop:** A critical component of this setup is the **AWS SNS (Simple Notification Service)** integration. Upon the alarm trigger, an automated email notification was successfully dispatched to the administrator.

Active alert state in the CloudWatch dashboard during a performance peak.

Granular metric view showing the exact moment the threshold was crossed.

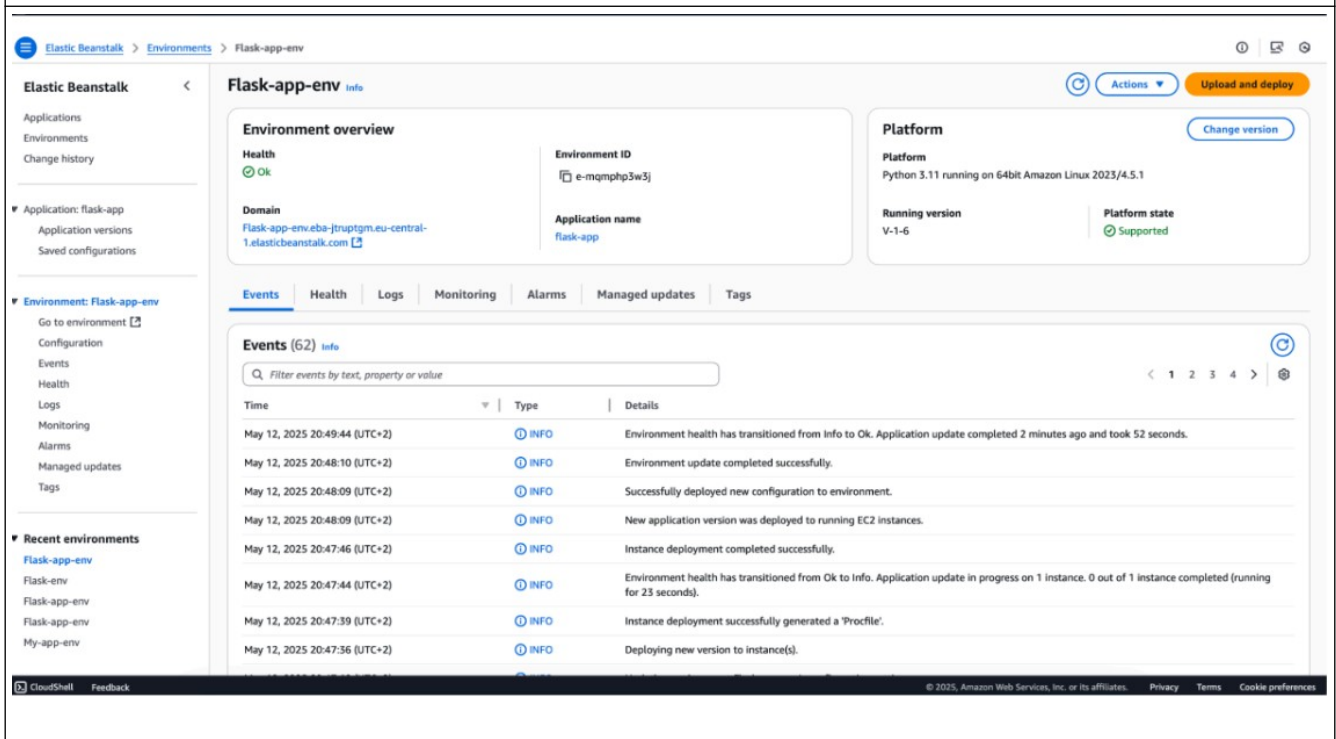
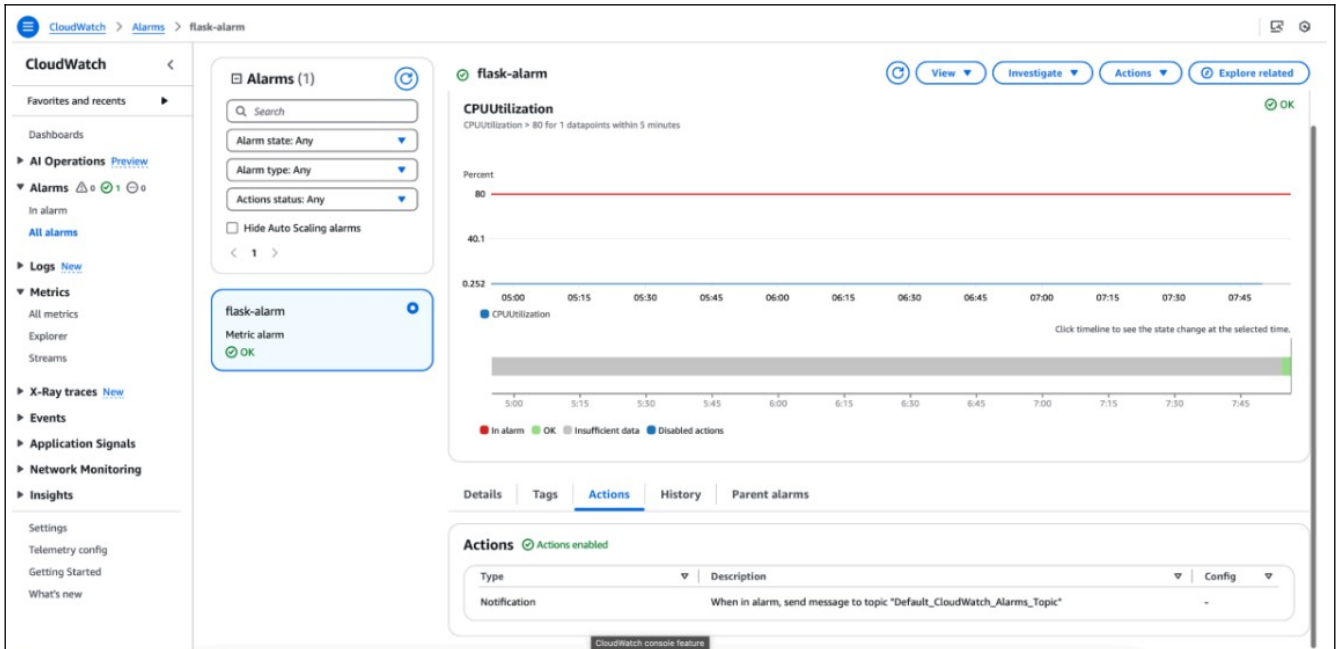
Evidence of the SNS-generated email alert, detailing the specific instance and metric that triggered the incident.

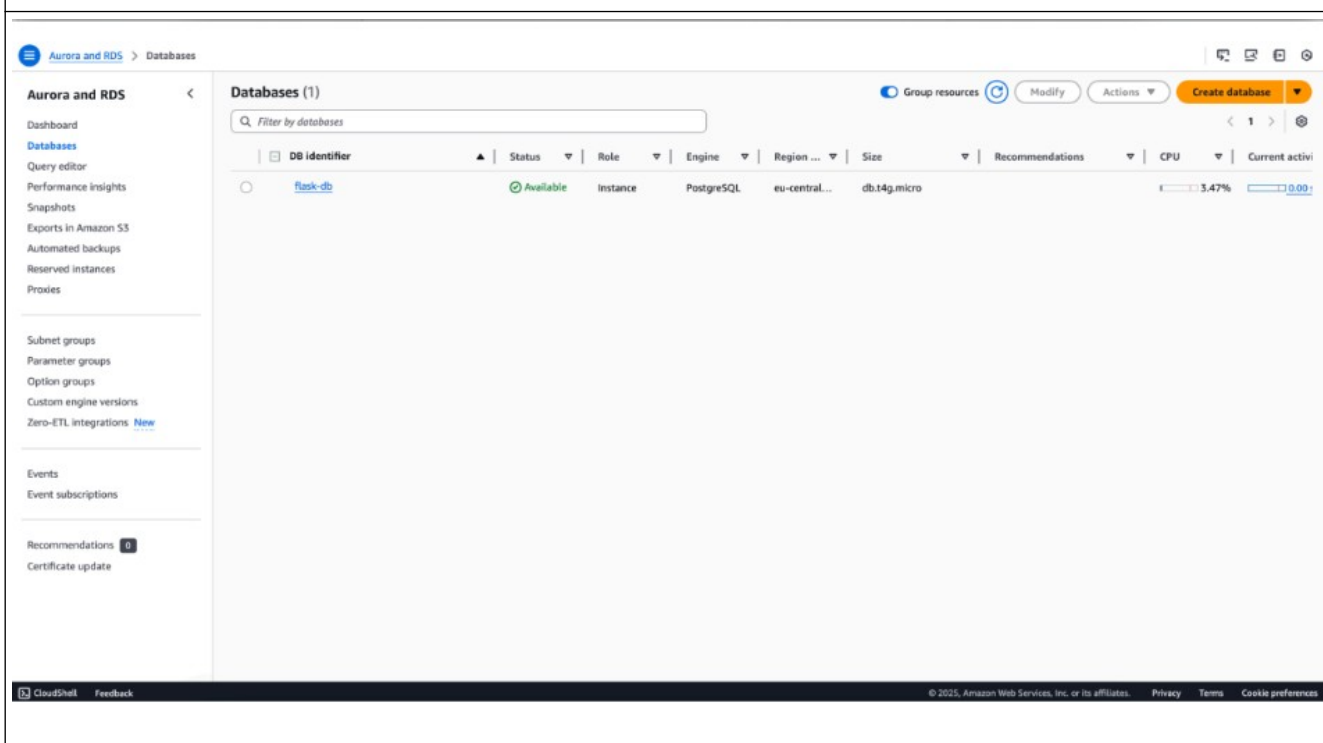
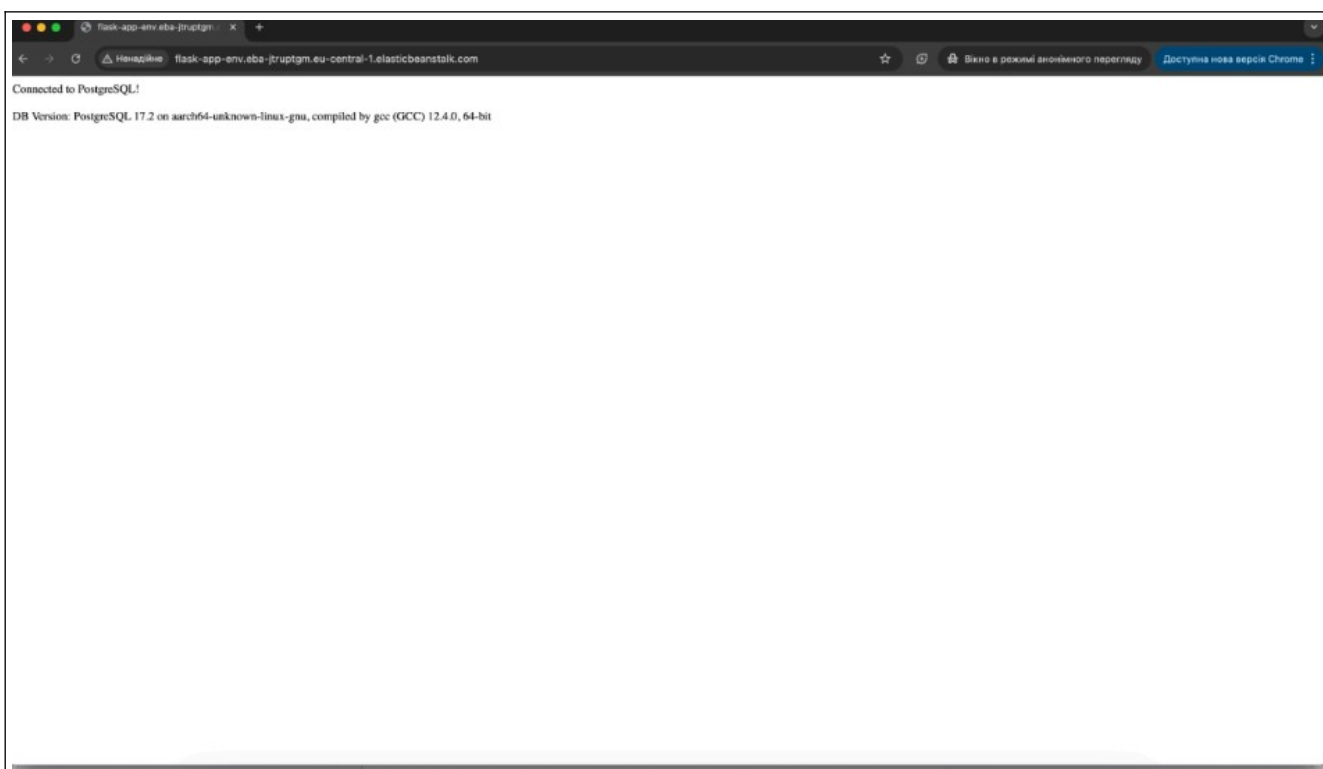
5.5 Granular Cost Management

The implementation of **AWS Budgets** includes precise thresholds to avoid unexpected billing:

- **Actual vs. Forecasted:** The system monitors both current spending and predictive trends, ensuring 360-degree financial visibility.
- **Multi-Tier Alerts:** Configured alerts at 85% and 100% of the budget provide a tiered response strategy for cost control.

Detailed breakdown of the active budget alerts and their current thresholds.





Elastic Beanstalk

Applications

Environments

Change history

▼ Application: flask-app

Application versions

Saved configurations

▼ Environment: Flask-app-env

Go to environment

Configuration

Events

Health

Logs

Monitoring

Alarms

Managed updates

Tags

▼ Recent environments

Flask-app-env

Flask-env

Flask-app-env

Flask-app-env

My-app-env

Instance traffic and scaling

Customize the capacity and scaling for your environment's instances. Select security groups to control instance traffic. Configure the software that runs on your environment's instances by setting platform-specific options.

Instances

IMDSv1

Deactivated

EC2 Security Groups

awsel-e-mqmph3w3j-stack-AWSEBSecurityGroup-lKwI7xd9wk6W

Capacity

Environment type

Load balanced

Min instances

1

Max instances

3

Fleet composition

On-Demand instances

On-demand base

0

On-demand above base

0

Capacity rebalancing

Deactivated

Scaling cooldown

360

Processor type

x86_64

Instance types

t3.micro, t3.small

AMI ID

ami-06a1d2c13bb6602dc

Availability Zones

Any

Metric

CPUUtilization

Statistic

Average

Unit

Percent

Period

5

Breach duration

5

Upper threshold

80

Scale up increment

1

Lower threshold

20

Scale down increment

-1

Load balancer

Load balancer visibility

public

Load balancer type

application

Load balancer is shared

false

Store logs

Deactivated

CloudShell

Feedback

© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

EC2

Dashboard

EC2 Global View

Events

▼ Instances

Instances

Instance Types

Launch Templates

Spot Requests

Savings Plans

Reserved Instances

Dedicated Hosts

Capacity Reservations

▼ Images

AMIs

AMI Catalog

▼ Elastic Block Store

Volumes

Snapshots

Lifecycle Manager

▼ Network & Security

Security Groups

Elastic IPs

Placement Groups

Key Pairs

Network Interfaces

▼ Load Balancing

Load balancers (1/1)

Elastic Load Balancing scales your load balancer capacity automatically in response to changes in incoming traffic.

Filter load balancers

☒	Name	DNS name	State	VPC ID	Availability Zones	Type	Date created
☒	awseb--AWSEB-qRunn...	awseb--AWSEB-qRunnMeakbai-110810930.eu-central-...	Active	vpc-0c5c917a15593bc0a	3 Availability Zones	application	May 13, 2025, 08:04 (U...

Load balancer: awseb--AWSEB-qRunnMeakbai

Details

Load balancer type

Application

Status

Active

VPC

vpc-0c5c917a15593bc0a

Load balancer IP address type

IPv4

Scheme

Internet-facing

Hosted zone

Z215JYR2R1TBD5

Availability Zones

subnet-0f1ce42690bf0b2c2 eu-central-1c (eucl-1-az1)

subnet-05db63e355e482ca7 eu-central-1b (eucl-1-az3)

subnet-05c3731bc2f9cfcc5 eu-central-1a (eucl-1-az2)

Date created

May 13, 2025, 08:04 (UTC+02:00)

CloudShell

Feedback

© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

Amazon SNS

Dashboard

Topics

Subscriptions

▼ Mobile

Push notifications

Text messaging (SMS)

Amazon SNS

Topics

Default_CloudWatch_Alarms_Topic

🔔 New Feature

Amazon SNS now supports High Throughput FIFO topics. [Learn more](#)

✕

Default_CloudWatch_Alarms_Topic

EditDeletePublish message

Details

Name

Default_CloudWatch_Alarms_Topic

ARN

arn:aws:sns:eu-central-1:279944202995:Default_CloudWatch_Alarms_Topic

Type

Standard

Display name

-

Topic owner

279944202995

Subscriptions

Access policy

Data protection policy

Delivery policy (HTTP/S)

Delivery status logging

Encryption

Tags

Integrations

Subscriptions (1)

Search

EditDeleteRequest confirmationConfirm subscriptionCreate subscription

ID

Endpoint

Status

Protocol

☐

b23c1008-3be5-4798-aabf-15c65d14c9e4

tschernomord@gmail.com

Confirmed

EMAIL

CloudShell

Feedback

© 2024, Amazon Web Services, Inc. or its affiliates.

Privacy

Terms

Cookie preferences

Billing and Cost Management

Budgets

Overview

Billing and Cost Management

Choose billing view [New](#)

Primary view

Home

Getting Started

Billing and Payments

Bills

Payments

Credits

Purchase Orders

Cost and Usage Analysis

Cost Explorer

Cost Explorer Saved Reports

Cost Anomaly Detection

Free Tier

Data Exports

Customer Carbon Footprint Tool

Cost Organization

Cost Categories

Cost Allocation Tags

Billing Conductor

Budgets and Planning

Budgets [New](#)

🟢 Your budget My Monthly Cost Budget has been created successfully. After creating a budget, it can take up to 24 hours to populate all of your spend data.

Submit feedback✕

Budgets (1) [Info](#)

Download CSVActionsCreate budget

Find a budget

Type - Show all budgets

☐

Name

Thresholds

Budget

Amount used

Forecasted amount

Current vs. budgeted

Forecasted vs. budgeted

☐

[My Monthly Cost Budget](#)

OK

\$5.00

-

-

0.00%

-

CloudShell

Feedback

© 2024, Amazon Web Services, Inc. or its affiliates.

Privacy

Terms

Cookie preferences

Billing and Cost Management

Budgets

Edit budget

Billing and Cost Management

Choose billing view New

Primary view

Home

Getting Started

Billing and Payments

Bills

Payments

Credits

Purchase Orders

Cost and Usage Analysis

Cost Explorer

Cost Explorer Saved Reports

Cost Anomaly Detection

Free Tier

Data Exports

Customer Carbon Footprint Tool

Cost Organization

Cost Categories

Cost Allocation Tags

Billing Conductor

Budgets and Planning

Budgets New

✔ Your budget My Monthly Cost Budget has been created successfully. After creating a budget, it can take up to 24 hours to populate all of your spend data.

Submit feedback

✕

Notification preferences

Select one or more notification preferences to receive alerts.

Email recipients

Specify the email recipients you want to notify when the threshold has exceeded.

tschernomord@gmail.com

Maximum number of email recipients is 10.

▶ Amazon SNS Alerts - Optional info

▶ AWS Chatbot Alerts

▼ Alert #2

Remove

Set alert threshold

Threshold

When should this alert be triggered?

100

% of budgeted amount

Trigger

How should this alert be triggered?

Forecasted

Summary: When your forecasted cost is greater than 100,00% (\$5.00) of your budgeted amount (\$5.00), the alert threshold will be exceeded.

Notification preferences

CloudShell

Feedback

☰

Billing and Cost Management > Budgets > Edit budget

Billing and Cost Management

Choose billing view New

Primary view

Home

Getting Started

Billing and Payments

Bills

Payments

Credits

Purchase Orders

Cost and Usage Analysis

Cost Explorer

Cost Explorer Saved Reports

Cost Anomaly Detection

Free Tier

Data Exports

Customer Carbon Footprint Tool

Cost Organization

Cost Categories

Cost Allocation Tags

Billing Conductor

Budgets and Planning

Budgets New

✔ Your budget My Monthly Cost Budget has been created successfully. After creating a budget, it can take up to 24 hours to populate all of your spend data.

Submit feedback

✕

▼ Alert #1 (0 actions attached)

Threshold

85%

Threshold measured against

Actual Costs

Add action

Email recipients

ttschernomord@gmail.com

Amazon SNS

Not configured

▼ Alert #2 (0 actions attached)

Threshold

100%

Threshold measured against

Forecasted Costs

Add action

Email recipients

ttschernomord@gmail.com

Amazon SNS

Not configured

▼ Alert #3 (0 actions attached)

Threshold

100%

Threshold measured against

Actual Costs

Add action

Email recipients

ttschernomord@gmail.com

Amazon SNS

Not configured

CloudShell Feedback

☰

Billing and Cost Management > Budgets > Budget details

Billing and Cost Management

Choose billing view New

Primary view

Home

Getting Started

Billing and Payments

Bills

Payments

Credits

Purchase Orders

Cost and Usage Analysis

Cost Explorer

Cost Explorer Saved Reports

Cost Anomaly Detection

Free Tier

Data Exports

Customer Carbon Footprint Tool

Cost Organization

Cost Categories

Cost Allocation Tags

Billing Conductor

Budgets and Planning

Budgets New

✔ Your budget My Monthly Cost Budget has been created successfully. After creating a budget, it can take up to 24 hours to populate all of your spend data.

Submit feedback

✕

Budget history Alerts Tags

Alerts (3) info

View details

Delete alert

Edit alerts

Actual cost > 85%

Definition

When your actual cost is greater than 85% (\$4.25) of your budgeted amount (\$5.00), the alert threshold will be exceeded.

Threshold

✔ Not exceeded

Actions

-

Forecasted cost > 100%

Definition

When your forecasted cost is greater than 100% (\$5.00) of your budgeted amount (\$5.00), the alert threshold will be exceeded.

Threshold

✔ Not exceeded

Actions

-

Actual cost > 100%

Definition

When your actual cost is greater than 100% (\$5.00) of your budgeted amount (\$5.00), the alert threshold will be exceeded.

Threshold

✔ Not exceeded

Actions

-

CloudShell Feedback

© 2025 Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

CloudWatch > Alarms

CloudWatch

Favorites and recents

Dashboards

AI Operations

Alarms

Logs

Metrics

X-Ray traces

Events

Application Signals

Network Monitoring

Insights

Settings

Telemetry config

Getting Started

What's new

Alarms (1)

Hide Auto Scaling alarms

Clear selection

Create composite alarm

Actions

Create alarm

Alarm state: In alarm

Alarm type: Any

Actions status: Any

Name

State

Last state update (Local)

Conditions

Actions

awseb-e-mqmp3w3j-stack-AWSEBCloudwatchAlarmHigh-AcJZg5dNjta

In alarm

2025-05-13 22:46:26

CPUUtilization > 80 for 1 datapoints within 5 minutes

Actions enabled

CloudWatch console feature

CloudShell

Feedback

© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

CloudWatch > Alarms > awseb-e-mqmp3w3j-stack-AWSEBCloudwatchAlarmHigh-AcJZg5dNjta

CloudWatch

Favorites and recents

Dashboards

AI Operations

Alarms

Logs

Metrics

X-Ray traces

Events

Application Signals

Network Monitoring

Insights

Settings

Telemetry config

Getting Started

What's new

Alarms (3)

Search

Alarm state: Any

Alarm type: Any

Actions status: Any

Hide Auto Scaling alarms

awseb-e-mqmp3w3j-stack-AWSEBCloudwatchAlarmHigh-AcJZg5dNjta

Metric alarm

In alarm

awseb-e-mqmp3w3j-stack-AWSEBCloudwatchAlarmLow-vw4zr1TFaKYG

Metric alarm

OK

flask-alarm

Metric alarm

OK

awseb-e-mqmp3w3j-stack-AWSEBCloudwatchAlarmHigh-AcJZg5dNjta

View

Investigate

Actions

Explore related

Graph

CPUUtilization

CPUUtilization > 80 for 1 datapoints within 5 minutes

Percent

100

50.1

0.263

19:45

20:00

20:15

20:30

20:45

21:00

21:15

21:30

21:45

22:00

22:15

22:30

22:45

Click timeline to see the state change at the selected time.

19:45

20:00

20:15

20:30

20:45

21:00

21:15

21:30

21:45

22:00

22:15

22:30

22:45

In alarm OK Insufficient data Disabled actions

CloudWatch console feature

Details

Tags

Actions

History

Parent alarms

Details

Name

State

Namespace

Datapoints to alarm

awseb-e-mqmp3w3j-stack-

In alarm

AWS/EC2

1 out of 3

CloudShell

Feedback

© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

ALARM: "awseb-e-mqmp3w3j-stack-AWSEBCloudwatchAlarmHigh-AcJZg5dNJta" in EU (Frankfurt)



AWS Notifications <no-reply@sns.amazonaws.com>
an mich ▾

13.05.2025, 22:47 ☆ ☺ ↶ ⋮

Zielsprache: Deutsch X

You are receiving this email because your Amazon CloudWatch Alarm "awseb-e-mqmp3w3j-stack-AWSEBCloudwatchAlarmHigh-AcJZg5dNJta" in the EU (Frankfurt) region has entered the ALARM state, because "Threshold Crossed: 1 out of the last 1 datapoints [80.93483812380903 (04/05/25 11:15:00)] was greater than the threshold (80.0) (minimum 1 datapoint for OK -> ALARM transition)." at "Sunday 04 May, 2025 11:20:22 UTC".

View this alarm in the AWS Management Console:
<https://eu-central-1.console.aws.amazon.com/cloudwatch/deeplink.js?region=eu-central-1#alarmsV2:alarm/awseb-e-mqmp3w3j-stack-AWSEBCloudwatchAlarmHigh-AcJZg5dNJta>

- Alarm Details:
- Name: awseb-e-mqmp3w3j-stack-AWSEBCloudwatchAlarmHigh-AcJZg5dNJta
 - Description: Alarm triggers when CPUUtilization exceeds 80% for 5 minutes
 - State Change: OK -> ALARM
 - Reason for State Change: Threshold Crossed: 1 out of the last 1 datapoints [80.93483812380903 (04/05/25 11:15:00)] was greater than the threshold (80.0) (minimum 1 datapoint for OK -> ALARM transition).
 - Timestamp: Sunday 04 May, 2025 11:20:22 UTC
 - AWS Account: 279944202995
 - Alarm Arn: arn:aws:cloudwatch:eu-central-1:279944202995:alarm:HighCPU-Alarm

Threshold:
- The alarm is in the ALARM state when the metric is GreaterThanThreshold 80.0 for at least 1 of the last 1 period(s) of 300 seconds.

- Monitored Metric:
- MetricNamespace: AWS/EC2
 - MetricName: CPUUtilization
 - Dimensions: [InstanceId = i-067015aff7e878aa6]
 - Period: 300 seconds
 - Statistic: Average
 - Unit: not specified
 - TreatMissingData: missing

- State Change Actions:
- OK: [arn:aws:sns:eu-central-1:279944202995:alarm]
 - INSUFFICIENT_DATA: